

GatePass

Event parking marketplace — project summary & schema | Draft v3

"GatePass" is a working name only — trademark (IP India), domain, and app-store name checks are still pending before it's final.

1. What this project is

A paid, two-sided marketplace for parking at public ticketed events in India — concerts, sports matches, exhibitions, fairs. Event organizers list available parking capacity at their venue; drivers discover and pay to reserve a spot in advance; the platform issues a digital QR pass for gate entry.

Why this segment, and not the others considered

Segment	Verdict	Reason
General P2P (any driveway, any city)	Rejected	Most Indian urban parking sits in apartment societies. Supreme Court (Nahalchand, 2010) and RERA treat allotted parking as a common area controlled by the RWA, not the flat owner's private property to rent out. Legally clean supply is much smaller than the UK/US model this idea was based on.
Private weddings / banquet halls	Rejected	A player (MetroPark+) already runs WhatsApp-based digital parking passes for this segment. More fundamentally, charging wedding guests to park is culturally unusual in India — the revenue model (commission on driver payment) doesn't fit; the incumbent monetizes as a B2B logistics tool sold to the venue, not a driver-paid marketplace.
Public ticketed events	Selected	Driver-paid parking is already a normal, accepted behaviour here (stadium parking, event parking). Supply is concentrated (one venue deal covers many spots). No dedicated India-specific competitor found for this exact niche at the time of research.

Builder context

Solo builder, bootstrapped (own time and limited budget), targeting the Indian market. Native mobile app chosen deliberately over a web-first MVP for credibility/portfolio reasons, accepting the extra build time and app-store overhead that comes with it (see Section 3).

2. Business model

Modelled on how Zomato/Swiggy settle with restaurants, not on a pure classifieds (OLX/Quikr) model, and not on a real-time payment-split marketplace (Razorpay Route) model:

- Driver pays the full amount into the platform's own account via a standard Razorpay Payment Gateway.
- Platform deducts its commission (kept conservative — 10–15% to start; Zomato/Swiggy's 25–35% effective take rate has driven restaurants to complain to the CCI and adopt zero-commission alternatives, which is a cautionary data point, not a benchmark to copy).

- Organizer is paid out on a periodic settlement (weekly, or per-event) with an itemized, per-booking breakdown — not a real-time split. This avoids the settlement-reconciliation disputes that Zomato/Swiggy restaurant partners commonly report.

Open item: whether this service category has a specific GST treatment (similar to Section 9(5) CGST for restaurant aggregators) has not been confirmed and needs a CA's input before launch. Business registration route: sole proprietorship + GST is expected to be sufficient for Razorpay KYC at this stage — full Pvt Ltd incorporation is not assumed to be necessary yet.

3. Tech stack

Layer	Choice	Why
Driver app	React Native (Expo, managed workflow)	Native chosen for credibility over web-first MVP. Expo over bare RN CLI for solo dev: handles build/app-store submission (EAS Build/Submit) and OTA JS updates without native reconfiguration.
Organizer dashboard	React (web)	Organizer works from a laptop to create listings and view settlement reports — no app-store distribution needed. Reuses React knowledge from the mobile app.
Backend	Node.js + Fastify + TypeScript	2–3x Express's throughput on JSON-heavy endpoints, built-in JSON Schema (Ajv) request validation, native TypeScript support. TypeScript chosen so booking/payment types can be shared across backend and both frontends, reducing integration bugs on amount/quantity fields.
DB access layer	Prisma ORM	Changed from Drizzle (v2). Prisma chosen for its migration tooling (<code>prisma migrate</code>), generated typed client, and relation handling. The one thing Drizzle was picked for — a precise conditional UPDATE for oversell prevention — Prisma still supports natively via <code>updateMany({ where: { ...bookedCount: { lte: total - qty } }, data: { bookedCount: { increment: qty } } })</code> , which compiles to a single atomic <code>UPDATE ... WHERE</code> . No escape-hatch raw query needed.
Database	PostgreSQL	Supports the atomic capacity-check pattern natively; no exotic extensions needed.
SMS / OTP delivery	MSG91	India-focused, cheapest reliable OTP delivery, free trial quota.
Auth	OTP (phone) + JWT sessions	See Section 5.
Payments	Razorpay Payment Gateway (not Route)	Full payment collected into the platform's account; payouts to organizers handled as periodic settlements (Section 2), not a real-time split.

Known gotcha: `react-native-razorpay` is a native module and will not run inside Expo Go. Payment testing needs an Expo Dev Client or an EAS development build — a common first surprise for Expo + Razorpay setups.

4. MVP architecture

- Single monolith service + Postgres. No microservices or Kubernetes at this stage — there is no scaling problem yet to justify the operational overhead.
- Oversell prevention is a single atomic conditional update on a capacity counter, rather than manual row locking.
- Idempotency keys on booking creation so retries from flaky mobile networks never create duplicate bookings.
- Digital QR pass: generated server-side on booking confirmation, delivered via WhatsApp/SMS and shown in-app; the venue's existing security staff verifies it visually — no scanner hardware or LPR investment for the MVP.
- Push notifications deferred: bookings are one-off per event, not daily-engagement usage, so WhatsApp/SMS confirmation covers it without a Firebase Cloud Messaging setup yet. The per-device FCM token is already stored (Section 5) so wiring FCM later needs no schema change.

Native app adds its own overhead — budget for it

- Apple Developer Program: \$99/year. Google Play: \$25 one-time.
- App Store review cycle (roughly 1–3 days, first submissions can be rejected) — build launch-date buffer around this.
- A CI/CD pipeline to trigger EAS Build/Submit (e.g. from GitHub Actions) is worth setting up early — also a good practitioner-level DevOps artifact for a portfolio.

Deferred until an actual problem forces it

Multi-party payment splitting (Razorpay Route) • geo-search across listings • horizontal scaling • real-time websocket availability • formal fraud/dispute tooling • push notifications (FCM) • native QR-scanning hardware/LPR.

5. Authentication & sessions (new in v3)

Phone-number OTP login, with a JWT that carries both identity and session reference. This is the piece the app's protected routes depend on.

Flow

1. `POST /auth/request-otp` — body: `phone`, `deviceId`, `deviceType`. A 6-digit code is generated server-side, stored in `otp_verifications` (5-minute expiry), and sent via MSG91.
2. `POST /auth/verify-otp` — body: `phone`, `otp`, `deviceId`, `deviceType`, plus optional `deviceName` / `fcmToken`. On match, the code is marked verified, the user is found-or-created by phone, and a `user_sessions` row is upserted per device. The response carries the JWT and the user.
3. `GET /auth/me` — returns the logged-in user.
4. `GET /auth/sessions` — lists all active devices for the user, flagging the current one.
5. `POST /auth/logout` — deletes the current session; the token stops working immediately.

6. DELETE /auth/sessions/:sessionId — remote logout of another device.

How a protected route reads the caller

The session service attaches the payload to the request, so handlers read the caller directly — no extra lookup:

```
const { userId, phone, role, sessionId, deviceId, deviceType } = request.user;
```

JWT payload

```
{ userId, phone, role, sessionId, deviceId, deviceType } // exp: 30 days
```

Sessions are validated on every request (the `user_sessions` row must still exist and not be expired), so logout and remote-logout take effect without a token blacklist. `last_active_at` is refreshed on each authenticated call.

Audit columns

`created_by` / `updated_by` (nullable FK → `users.id`) are added to `listings`, `bookings`, `payments`, and `settlements`, populated from `request.user.userId` on write. This gives a per-row "who did it" trail without a separate audit-log table at MVP. The columns exist on all four tables; the create routes currently populate them on `listings` — the booking/payment/settlement write paths still need to set them.

MSG91 integration notes (learned the hard way)

- Mobile number must be sent with country code (`91XXXXXXXXXX`) — a bare 10-digit number will fail or misroute.
- The SMS template must contain the `##OTP##` variable placeholder, matched to the template ID.
- Until a key is configured, dev mode logs the OTP and can return it in the response (`SHOW_OTP_IN_RESPONSE=true`) so the flow is testable without spending SMS credit.

Still to harden before launch: per-phone rate limiting on OTP requests, a cap on wrong-OTP attempts (a 6-digit code with unlimited tries is brute-forceable), digits-only phone validation, and a real `JWT_SECRET`.

6. Database schema

Naming note: the original v2 spec below used snake_case table/column names (e.g. `parking_capacity`, `created_at`). The current Prisma schema uses Prisma's default naming — PascalCase models/tables (`User`, `Listing`) and camelCase columns (`createdAt`, `organizerId`). The names below are kept in the v2 snake_case style for readability; add `@map` / `@@map` to the Prisma models if DB-level snake_case is wanted. Domain naming is already aligned with v2: `events` → `listings`, with `listing_type` (only `event` in use today) so a future recurring/commercial supply type needs no rename. The recurring case is not built yet: it would be modelled as auto-generated daily capacity rows so the same atomic counter logic keeps working, with no new concurrency design.

users

Column	Type	Notes
id	uuid, PK	
phone	varchar, unique, not null	Primary auth identifier
name	varchar	
role	enum(driver, organizer, admin)	
gst_number	varchar, nullable	Organizer only
bank_account_id	varchar, nullable	Razorpay Fund Account ref, for payouts
created_at / updated_at	timestamp	

Open design question: `gst_number` and `bank_account_id` are always null for drivers. Fine for MVP; consider splitting into a separate `organizer_profiles` table if organizer-specific fields grow.

otp_verifications (new in v3)

Column	Type	Notes
id	uuid, PK	
phone	varchar, indexed	Looked up on verify
code	varchar	6-digit OTP
device_id	varchar, nullable	Which device requested it
device_type	enum(ios, android, web, other), nullable	
expires_at	timestamp	5 minutes from creation
verified	boolean, default false	Set true on successful verify
created_at	timestamp	

user_sessions (new in v3)

Column	Type	Notes
id	uuid, PK	Referenced by the JWT as <code>sessionId</code>
user_id	uuid, FK → users.id, indexed	Cascade delete
device_id	varchar, indexed	Unique together with user_id
device_type	enum(ios, android, web, other), default other	
device_name	varchar, nullable	e.g. "iPhone 15"
fcm_token	varchar, nullable	Reserved for future push; no schema change needed later

Column	Type	Notes
last_active_at	timestamp	Refreshed on each authenticated request
expires_at	timestamp	30 days from issue
created_at	timestamp	

listings (renamed from events)

Column	Type	Notes
id	uuid, PK	
organizer_id	uuid, FK → users.id	Indexed
listing_type	enum(event, recurring, commercial)	Only 'event' used today
name	varchar	
venue_name	varchar	
latitude / longitude	decimal	
event_date	date	Applies to listing_type = event
status	enum(draft, published, ongoing, completed, cancelled)	
created_by / updated_by	uuid, FK → users.id, nullable	Audit (new in v3)

parking_capacity

Column	Type	Notes
id	uuid, PK	
listing_id	uuid, FK → listings.id	Renamed from event_id
vehicle_type	enum(car, bike, other)	Unique with listing_id
total_capacity	int	
booked_count	int, default 0	Target of the atomic capacity-check update
price	decimal	

bookings

Column	Type	Notes
id	uuid, PK	
parking_capacity_id	uuid, FK	
driver_id	uuid, FK → users.id	
vehicle_number	varchar	

Column	Type	Notes
quantity	int	
amount	decimal	Price snapshot at booking time
status	enum(pending, confirmed, cancelled, completed, no_show)	
idempotency_key	varchar, unique	Safe retries
qr_token	varchar, unique	Digital pass
created_by / updated_by	uuid, FK → users.id, nullable	Audit (new in v3)

payments

Kept separate from bookings because payment status and booking status are two different state machines — a booking can be pending while a payment is created but not yet captured.

Column	Type	Notes
id	uuid, PK	
booking_id	uuid, FK, unique	
razorpay_order_id / razorpay_payment_id	varchar	
amount	decimal	
status	enum(created, captured, failed, refunded)	
created_by / updated_by	uuid, FK → users.id, nullable	Audit (new in v3)

settlements

Column	Type	Notes
id	uuid, PK	
organizer_id	uuid, FK	
period_start / period_end	date	
gross_amount / commission_amount / net_payable	decimal	
status	enum(pending, processing, paid, disputed)	
created_by / updated_by	uuid, FK → users.id, nullable	Audit (new in v3)

settlement_items

Per-booking breakdown inside a settlement, so the organizer gets an itemized report instead of a single lump figure — this is what Zomato/Swiggy restaurant partners commonly say they don't get, leading to reconciliation disputes.

Column	Type	Notes
id	uuid, PK	
settlement_id	uuid, FK	
booking_id	uuid, FK	
amount / commission_deducted	decimal	

Indexes to add

`listings.organizer_id` • `parking_capacity.listing_id` • `bookings.parking_capacity_id` •
`bookings.driver_id` • `bookings.idempotency_key` (unique) • `payments.booking_id` •
`settlements.organizer_id` • `settlement_items.settlement_id` • `otp_verifications.phone` •
`user_sessions.user_id` • `user_sessions.device_id`

7. Migrations & local setup (new in v3)

Migrations are committed one per model/concern rather than as a single blob, so each schema change is reviewable in isolation:

Migration	Adds
0001_enums	All enums (<code>Role</code> , <code>ListingType</code> , <code>ListingStatus</code> , <code>VehicleType</code> , <code>BookingStatus</code> , <code>PaymentStatus</code> , <code>SettlementStatus</code>)
0002_user	<code>User</code> table
0003_device_otp	<code>DeviceType</code> enum + <code>OtpVerification</code>
0004_user_session	<code>UserSession</code> (device tracking)
0005_base_tables	<code>Listing</code> , <code>ParkingCapacity</code> , <code>Booking</code> , <code>Payment</code> , <code>Settlement</code> , <code>SettlementItem</code> — with <code>createdBy</code> / <code>updatedAt</code> built in from the start

Postgres runs in Docker with a named volume

`docker-compose.yml` runs `postgres:16-alpine` with a named volume `pgdata` mounted at `/var/lib/postgresql/data`. Data survives container restarts and `docker compose down`; only `docker compose down -v` wipes it. The backend service waits on a `pg_isready` healthcheck before starting and runs `prisma migrate deploy` on boot.

```
# start just the database for local dev
docker compose up -d db

# apply migrations then generate the typed client
cd be && npx prisma migrate deploy && npx prisma generate
```

`schema.prisma` stays the source of truth for both the database structure (via migrations) and the TypeScript types (via `prisma generate`) — a model must be added there before a table can exist.

8. Change log

v2 → v3

- ORM switched from Drizzle to Prisma (Section 3); the atomic oversell query maps to Prisma's `updateMany` with a guarded `where`, no raw SQL needed.
 - Added phone-OTP authentication with 30-day JWT sessions and per-device session tracking (Section 5) — new `otp_verifications` and `user_sessions` tables, new `DeviceType` enum.
 - Added `created_by` / `updated_by` audit columns to `listings`, `bookings`, `payments`, `settlements`.
 - Documented the migration workflow and the Docker Postgres volume (Section 7).
 - Recorded MSG91 integration requirements and the outstanding auth hardening items (Section 5).
-

This is a working draft reflecting decisions made to date — expect the schema to evolve as the booking-flow and settlement logic are built out in detail.